Dezyne E'cole College

Business • Design • Computers

HACKATHON 2026

Resilience Report

HealTrack — Self-Healing Order Tracking and Delivery Management System

SUBMITTED BY

Yash Parashar

BCA Final Year Student

October 2026

RESILIENCE REPORT

HealTrack

Self-healing order tracking and delivery management

Measurements: 23 Oct 2026
Node.js 22 · MongoDB Atlas · OSRM · PM2
Report prepared: 26 Oct 2026

1. Summary

HealTrack creates delivery orders, calculates road routes and tracks a rider on a live map. This report describes how the system detects and heals five kinds of failure, using a built-in **Chaos Lab** that injects controlled, time-limited faults and a **Self-Healing Portal** that shows each incident's lifecycle. Every figure below was measured from real runs against the running system, not estimated.

An incident is only marked **RECOVERED** after a verification step passes (for example, a fresh rider location read back from the database). Otherwise it is reported as **DEGRADED** or **FAILED**.

Chaos scenarios passing **6 / 6** 5 failure types + offline variant	Rider stall: time to detect **4.8–6.5 s** was: never detected	Rider stall: time to recover **3.8–5.2 s** verified fresh location	API crash: downtime **5.1–6.1 s** PM2 restart + health check
Routing outage: recovery **11.9–12.8 s** incl. 10 s breaker cooldown	DB write failure: recovery **1.4–3.1 s** 2 retries + read-back	Incidents stuck "in progress" **0** was: 17 of 33 (52%)	Recovery verified with evidence **100%** was: 0% of incidents

2. Architecture



3. Failure table

Recovery times are measured from persisted incidents: **TTD** = detected − fault start; **TTR** = verified recovery − detected. Ranges cover every run of that scenario on 23 Oct 2026; *n* = number of runs.

FAILURE	HOW IT IS DETECTED	HOW IT HEALS	HOW RECOVERY IS VERIFIED	MEASURED
Rider tracking stalls worker freezes, no new positions	Monitor every 2 s; location older than 7.5 s (2.5× the 3 s update interval) while tracking is active.	Keeps the last valid position (shown as stale to the customer), diagnoses "worker ticking but not writing", restarts the worker from saved progress. Up to 3 attempts with 1 s / 2 s back-off.	New location timestamp after the restart, valid coordinates, progress not reset, worker running.	TTD 4.8–6.5 s TTR 3.8–5.2 s n = 5 (TTR 4), 1 attempt
Rider device offline writes fail for 14 s	Same staleness monitor; consecutive write errors recorded on the order.	Diagnoses "location writes failing", restarts the worker with back-off until updates succeed. If attempts run out: DEGRADED, last position kept, monitor keeps checking.	Same read-back checks as above.	TTD 4.9 s TTR 13.4 s n = 1, 2 attempts
Routing (OSRM) outage every route call fails for 5 s	First failed route request (error classified: timeout, 5xx, invalid response, outage).	3 attempts with 250 / 500 ms back-off → circuit breaker OPEN → last-known route served and marked **DEGRADED**; calls skipped while open; after 10 s cooldown a half-open probe tries live routing.	Probe returns a live route with ≥ 2 points and a valid distance; breaker back to CLOSED.	TTD < 0.1 s TTR 11.9–12.8 s n = 3, 2 retries
Database write failure 2 transient errors	Write throws; error classified as retryable while the connection is still up.	Up to 4 attempts with 200 / 400 / 800 ms back-off. Non-retryable errors are not retried. With 5 injected errors: FAILED, nothing partial written.	Read-after-write: the probe document is read back with a matching token.	TTD ≈ 0 s TTR 1.4–3.1 s n = 5, 2 retries
Invalid tracking data lat 999, "abc", lng −500, null	Validation on every location update (numeric, latitude −90…90, longitude −180…180).	Update rejected with HTTP 400; stored location, status and progress untouched; repeated bad updates within 60 s are grouped into one incident.	Order re-read: location still valid, not overwritten, progress not lower.	4 / 4 rejected in every run TTR 0.9 s (n = 1)
API process crash process exits with code 1	Incident saved before the exit; PM2 sees the process die.	PM2 restarts the process; on startup the API resumes rider workers from saved progress and closes interrupted test runs.	New process ID, MongoDB ping OK, own `GET /api/health` returns 200.	Downtime / TTR 5.1–6.1 s n = 2

Incident lifecycle as stored (one incident ID throughout)



Example, rider stall (incident 6adb613f…ab1e): detected +0.0 s, diagnosing +0.8 s, recovering +1.6 s, verifying +4.5 s, recovered +4.7 s. A routing outage passes through DEGRADED (cached route) before VERIFYING; an exhausted DB retry ends at FAILED.

4. Before and after

"Before" is the project as found on 23 Oct 2026: its stored incidents and its code paths. "After" is the same project with the healing engine, measured the same day.

Stored incident data

MEASURE	BEFORE	AFTER
Incidents compared	33 (all on record)	28 (created by the new engine)
Stuck "in progress" with no way to resolve	17 (52%)	0
Incidents with a stage-by-stage timeline	0	28 of 28
Recoveries backed by a recorded verification	0 of 14	24 of 24
Outcomes	14 recovered*, 2 failed, 17 open	24 recovered, 4 failed**
Recovered ÷ all incidents (dashboard formula before)	42.4%	85.7%
Recovered, excluding deliberate negative tests	—	24 of 24 = 100%

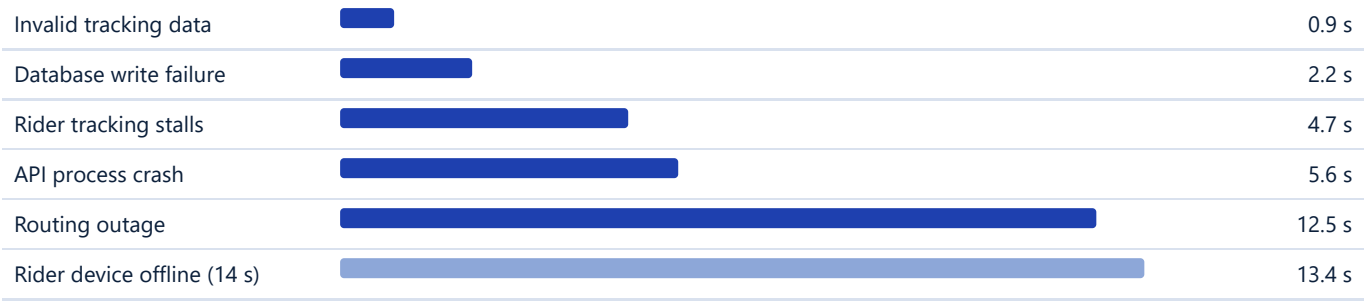
* 25 of the 33 "before" incidents came from an old routing simulator that marked incidents recovered without checking anything and left 17 open forever; those 25 were removed from the database after this comparison (a backup was kept). ** All 4 failures are deliberate negative tests: database retries were exhausted on purpose and the system correctly reported FAILED instead of claiming recovery.

Behaviour per failure

FAILURE	BEFORE	AFTER (MEASURED)
Rider tracking stalls	Not detected Map kept showing the old position; no incident, no recovery.	Healed Detected in 4.8–6.5 s, tracking restored and verified in 3.8–5.2 s.
API restart	Riders lost Simulation timers lived in memory; every moving rider froze until restarted by hand (and restarting reset progress to 0).	Resumed Riders resume from saved progress (test: route point 2 → 4 across the restart), no false incident.
API crash	Unverified No supervisor installed; on the next manual start every crash incident was marked recovered without a health check.	Healed PM2 restart, DB ping + health check, 5.1–6.1 s downtime.
Routing outage	Partial Retry, breaker and cache existed, but incidents went straight to "recovering" and some never closed.	Healed Same breaker, now DEGRADED while on cache and RECOVERED only after a live probe; 11.9–12.8 s.
Database write failure	No handling Request failed with HTTP 500; nothing recorded.	Healed Bounded retry + read-back, 1.4–3.1 s; honest FAILED when retries run out.
Invalid tracking data	Rejected silently HTTP 400, no record, nothing verified.	Protected Rejected, data integrity verified, one grouped incident.
Chaos controls	Unprotected Any client could switch routing into permanent failure.	Guarded Off by default, never in production, localhost only, faults auto-expire (≤ 60 s).

4b. Measured recovery time by failure

Median of runs per failure type.



Routing recovery is dominated by the deliberate 10 s breaker cooldown. Device-offline recovery depends on how long the fault lasts (14 s in this test); the lighter bar marks a single run.

5. How the numbers were measured

- Each scenario was started from the Chaos Lab (or its API) against the running system with real MongoDB Atlas and the public OSRM service. Faults are injected in test-only wrappers: no stored data is corrupted, and database faults are raised before MongoDB is contacted.
- Timings come from timestamps written to the incident record at each lifecycle step, not from the browser.
- Automated smoke test (`npm run test:smoke -- --osrm --negative`): 12 / 12 checks passed, then 10 / 10 in an immediate second run; crash test run twice under PM2; browser flow checked from the Chaos Lab button to the same incident in the Portal and the customer's "Tracking restored" notice.

6. Known limits

LIMIT	IMPACT AND MITIGATION
Simulated rider	Locations come from a route simulation, not real GPS. If a real device stops sending, the backend cannot obtain a position; HealTrack can only show the last confirmed location as stale.
Single process, in-memory state	Circuit-breaker state, route cache and armed faults live in one Node.js process. A restart clears them (riders and incidents are persisted and recover). Running several API instances would need shared state (e.g. Redis).
Rule-based diagnosis	Root causes are chosen by fixed rules from observed evidence, with an allow-list of recovery actions. There is no AI diagnosis.
Untested outcomes	Tracking ending DEGRADED (device offline longer than all recovery attempts) and routing staying DEGRADED (real OSRM down during the probe) are implemented but were not triggered: the bounded test faults always ended first.
Database scenario scope	Faults target a test-only probe collection. Rider location writes use the same guarded path, but no scenario injects faults there yet. A full Atlas outage was not simulated.
PM2 and access control	Crash recovery needs PM2 (with plain `npm start` the scenario reports BLOCKED). Chaos controls are protected by an environment flag, localhost-only access and an optional token; the app has no user login.
External dependencies	Public OSRM and Nominatim have rate limits and no service guarantee. Only routes already calculated are available from cache during an outage; new destinations fail until routing returns.
Detection speed bound	Detection is bounded by the 7.5 s staleness threshold plus the 2 s monitor interval; a lower threshold would cause false alarms on slow networks.

7. Next steps

Add scenarios for failed rider-location writes and address-search (Nominatim) outages; move breaker and cache state to a shared store for multiple API instances; add unit tests for the lifecycle engine and fault injector; measure detection with real rider devices on real networks.